Waterloo Rocketry 2025 - Aurora

Controls subsystem

Roll Control Design and Simulation of a Canard-controlled Sounding Rocket

Finn Breuer [1] Joe Dolina [2] Tristan Yan-Klassen [3]

February 11, 2025

[1]Technische Universität Hamburg, finn.breuer@tuhh.de
[2]University of Waterloo
[3]University of Waterloo

Contents

I Estimator & Controller **1**

1 Modelling . . . 1
- 1.1 Kinematics and dynamics . . . 2
- 1.2 Measurements . . . 5
- 1.3 Atmosphere and air data . . . 7
- 1.4 Forces and moments . . . 9
- 1.5 Estimator model equations . . . 12
- 1.6 Controller model equations . . . 14

2 Estimator . . . 15
- 2.1 Filter algorithm . . . 15
- 2.2 Optimal estimation theory . . . 17
- 2.3 Filter tuning . . . 17
- 2.4 Implementation notes . . . 18
- 2.5 Initializor . . . 19
- 2.6 Hard calibration . . . 19

3 Controller . . . 20
- 3.1 Controller roll model . . . 20
- 3.2 Control law and tuning . . . 21
- 3.3 Optimal control theory . . . 22
- 3.4 Gain scheduling . . . 22
- 3.5 Response analysis . . . 23

4 Numerical methods . . . 25
- 4.1 Numerical Jacobian computation . . . 25
- 4.2 Numerical Integration methods . . . 25

5 Implementation . . . 26
- 5.1 Overview . . . 26
- 5.2 Estimator . . . 26
- 5.3 Model functions . . . 27
- 5.4 Controller . . . 27

6 Symbols . . . 27

II Flight Simulation **29**

7 Plant Model . . . 29
- 7.1 Background . . . 29
- 7.2 Theory . . . 30
- 7.3 Implementation . . . 30

8 Simulation results . . . 33
- 8.1 Rocket flight . . . 33

8.2 Estimator in the loop 33
8.3 Controller in the loop 33
8.4 Full closed-loop 33

References **34**

Chapter I

Estimator and Controller Design

Finn Breuer

In this chapter, the rocket flight is modelled, and a control loop consisting of state estimation and a controller is designed using this model. The state estimation is realized with an EKF, and a gain scheduled LQR is used as feedback control.

1 Modelling

Dynamical models of the rocket flight are needed for both the controller synthesis and the estimator algorithm. The rocket model is made up of a system of algebraic and differential equations [1, 2, 3], represented in *state-space* as [2, 3]

$$\dot{x} = f(t, x, u) \tag{I.1}$$
$$y = h(t, x, u) \tag{I.2}$$

with the state x, measurement output y, and control input u. The state (or dynamics) equation f and the output equation h can be nonlinear functions, and may also depend on the time t[1]. The equations are vector-valued, and the variables x, u, y are vectors.

Linear time-invariant (LTI) state-space models can be derived by linearizing the equations f and h around some point (t_0, x_0, u_0), which results in the equations

$$\dot{x} = A(x - x_0) + B(u - u_0) \tag{I.3}$$
$$y = C(x - x_0) + D(u - u_0) \tag{I.4}$$

where the matrices A, C are the Jacobians of f, h with respect to the state x, and similar C, D are Jacobians with respect to the input u.

The state x describes a collection of variables, which describe the dynamics of the system in a collection of first order differential equations. [2] These states do not have to have

[1] The equations will turn out to not depend explicitly on time, i.e. the differential equations I.1 may be treated as autonomous.

[2] For example: a mass-spring-damper system has two state variables, position and velocity. These describe the energy stores of the system, potential of the deformed spring and kinetic of the moving mass.

a physical meaning [3], but they will in this case as they describe the system well. From textbooks [1, 3, 4], lectures [5], papers [6, 7], the industry standard for choosing states of aeronautical vehicles is using attitude angles, attitude rates, velocity, and position. The input signal u describes how external sources influence the dynamics (and the measurement). This describes behaviour which is not captured by the state alone, for example actuators acting on the system. The output signal y describes how a certain state influences the environment. This mostly encompasses measurements taken from the system by sensors, or the variables of interest for a control loop.

In this section the "rocket-flight" system will be modelled. "Modelling" means finding the underlying equations which describe the behaviour of the rocket in flight, its sensors and actuators, the environment, etc., and making assumptions and simplifications to ease the design of the control loop. As the famous (and often repeated by my professors) quote goes

> "All models are wrong, but some are useful" - George Box

1.1 Kinematics and dynamics

Coordinate frames

The inertial frame f^G is assumed to be the earth-flat geographic frame, with an up-crossrange-downrange orientation (this is more useful than up-north-west). The origin is located below the launch rail, with zero altitude at sea level.

The body-fixed frame f^B is oriented along the main inertia axes of the rocket body, namely forward-starboard-rail (x^+ is forward, and z^+ points toward the launch rail, downrange at launch), as shown in Figure I.1. The origin of the body-fixed frame is (as typical) located at the body center of gravity.

The sensor frames f^S are aboard the rocket, but rotated and translated against the body frame.

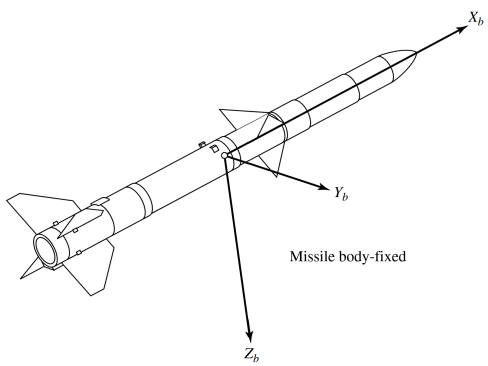


Figure I.1: Diagram of the body-fixed frame f^B, taken from [8]. (Here a different convention is used to denote the axes, X_b rather than x^B).

The rotational transformation of a vector r between the frames is given by

$$r^G = Sr^B \qquad r^B = S^T r^G \tag{I.5}$$

$$r^B = S_i r^S \qquad r^S = S_i^T r^B \tag{I.6}$$

[3] For example, one could describe a system with its input-output relationship, without assigning a meaning to the variables of the differential equations.

with the rotation matrix S (see Equation I.14), and the rotation matrix S_i for the i-th sensor frame. The frames f^G and f^B are shown in Figure I.2 with their rotation against each other. To simplify the following notations, the velocities, the angular rates, and the external forces and torques are given in the body-fixed frame, and the index ^{B} is omitted.

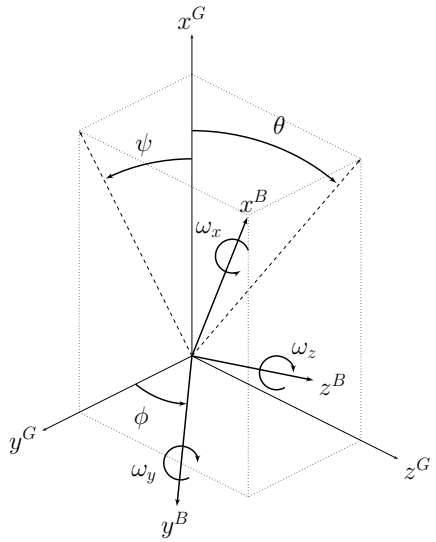


Figure I.2: Diagram of the attitude coordinate frames with Euler angles and body rates.

Attitude with Quaternions

In aeronautics Euler angles are often used for control design. However, the kinematics of the Euler angles show singularities at certain orientations, and the notation with large rotation matrices from sines and cosines is cumbersome. For this reason the attitude of the rocket will be described by quaternions, as

> "these differential equations are a joy to implement, because they are linear, have no singularities, and number only four" - Peter Zipfel [1]

The Quaternion is defined as

$$q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T = \begin{bmatrix} q_w \\ q_v \end{bmatrix} \tag{I.7}$$

which consists of four variables, often names as the real part q_w, and the vector part q_v.[4] The attitude quaternion is normed to $|q| = q^T q = 1$ to describe rotations. This unit constraint has to be enforced in the filter process, e.g. by norming q after integration [9].

Euler's rotation theorem states that every rotation sequence (or attitude difference) in 3D space can be expressed as a single rotation with angle φ around a single axis n (unit vector) [1]. The quaternion can be understood as that rotation, with

$$q_w = \cos(\frac{\varphi}{2}) \qquad\qquad q_v = n\sin(\frac{\varphi}{2}) \tag{I.8}$$

[4]Formally, the quaternion is a four-dimensional complex number, instead of $a+\mathrm{i}b$ it is $q_w+\mathrm{i}q_x+\mathrm{j}q_y+\mathrm{k}q_z$, where all variable are orthogonal and $\mathrm{i}^2 = \mathrm{j}^2 = \mathrm{k}^2 = \mathrm{ijk} = -1$. This will not really be relevant for the mechanics, but explains some weirdness of the calculations.

The inverse of a unit quaternion is simply [3]

$$q^{-1} = \begin{bmatrix} q_w \\ q_v \end{bmatrix}^{-1} = \begin{bmatrix} q_w \\ -q_v \end{bmatrix} \tag{I.9}$$

To rotate one attitude to another attitude, the attitude quaternions are multiplied using the Hamiltonian product $*$. This can also be applied to rotate vectors, but the rotation matrix S (Equation I.14) is computationally more efficient. Two quaternions q, r are multiplied to quaternion p as [3]

$$p = q * r = \tilde{Q} \cdot r \qquad \tilde{Q} = \begin{bmatrix} q_w & -q_x & -q_y & -q_z \\ q_x & q_w & -q_z & q_y \\ q_y & q_z & q_w & -q_x \\ q_z & -q_y & q_x & q_w \end{bmatrix} \tag{I.10}$$

with the matrix $\tilde{Q}$ stemming from the Hamiltonian product [3]. To note: this multiplication is not commutative ($q * r \neq r * q$).

The time derivative of the quaternion attitude, with angular rates ω described in the body-fixed frame, is [1, 9, 3]

$$\dot{q} \;=\; \frac{1}{2}\tilde{\Omega} \cdot q \;=\; \frac{1}{2}\tilde{Q} \cdot \begin{bmatrix} 0 \\ \omega \end{bmatrix} \qquad \tilde{\Omega} = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix} \tag{I.11}$$

Time integration leads to the discrete dynamics of a quaternion attitude, described by the rotation with an incremental attitude difference [9, 3]

$$q(t + dt) = q(t) * dq(t, dt) \qquad \Longrightarrow \qquad q_{k+1} = \tilde{Q}_k \cdot dq_k \tag{I.12}$$

where for constant angular velocities ω in this short time frame dt, the incremental attitude quaternion dq is [9]

$$dq(t, dt) = \begin{bmatrix} \cos(\frac{d\varphi}{2}) \\ dn \sin(\frac{d\varphi}{2}) \end{bmatrix} \qquad d\varphi = \|\omega(t)\| dt \qquad dn = \frac{\omega(t)}{\|\omega(t)\|} \tag{I.13}$$

This is the approximate (truncated Taylor series) closed-form solution of the differential equation I.11. Its explicit, first order, and maintains the unit constraint, thus this form will be used for time integration of the quaternion, rather than explicit Euler for the other derivatives (see Section 4.2).

The rotation matrix is used to transform a vector from one attitude to another, i.e. for the transformation from the body-fixed frame to the inertial frame. It can be computed directly from the quaternion [1, 3]. The rotation matrix $S = S_{G \to B}$ (or often $S = S_G^B$) transforms a vector from the ground frame G to the body-fixed frame B, and is computed as

$$S = 2 q_v q_v^T + (q_w^2 - q_v^T q_v) I_3 - 2 q_w \tilde{q}_v \tag{I.14}$$

The inverse transformation, i.e. from a vector in B to a vector in G, is performed by inverting the transformation matrix. Luckily, the rotation matrix is always orthogonal, so $S^{-1} = S^T$, and thus $S_B^G = S^T$.

The Euler angles *yaw* ψ, *pitch* θ, and *roll* ϕ can be recovered from the rotation matrix elements (or directly from the quaternions). For the rotation sequence yaw $\rightarrow$ pitch $\rightarrow$ roll (order $z \rightarrow y \rightarrow x$, rotation around the respective new body axes) the Euler angles are

$$\psi = \text{atan2}(S_{21}, S_{11}) \qquad \theta = \arcsin(-S_{31}) \qquad \phi = \text{atan2}(S_{23}, S_{33}) \tag{I.15}$$

For initialization, the quaternion is set according to Equation I.8, where φ the launch angle, and n indicates the launch orientation (pitch or yaw) [5]. A typical initialization would be φ = launch angle, and $\boldsymbol{d} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}^T$, i.e. the rocket is pitched downrange, and both yaw and roll are defined zero.

Relative motion

Using Newton's second law, the acceleration in the body-fixed frame is [1, 3]

$$\dot{v} = \mathcal{F}/m - 2\,\omega \times v + Sg^G \tag{I.16}$$

which accounts for the derivation in the rotating body-fixed frame with the transport term $-\,\omega \times v$, and the vector of external forces $\mathcal{F}$ (in the body-fixed frame). The gravity vector $g^G = \begin{bmatrix} -g_0 & 0 & 0 \end{bmatrix}^T$ is transformed to the body-fixed frame to account for the gravitational acceleration (magnitude $g_0 \approx 9.81 \frac{\text{m}}{\text{s}^2}$). Note: gravity is thus not a part of the force $\mathcal{F}$.

The velocity in the inertial (geographic) frame is [1]

$$\dot{r}^G = S^T v \tag{I.17}$$

With Euler's law, the angular acceleration is [1, 10]

$$\dot{\omega} = J^{-1}(\mathcal{T} - \omega \times J\omega) \tag{I.18}$$

which includes the external torques (or moments) $\mathcal{T}$ (described in the body-fixed frame) and the gyroscopic moment. The inertia tensor J is determined from CAD or experimentally. If the coupling inertia moments are very small, they may be neglected to fill the tensor diagonally as $J = \text{diag}\begin{bmatrix} J_x & J_y & J_z \end{bmatrix}$, where in a rocket cruciform typically $J_x < J_y = J_z$.

1.2 Measurements

All sensors are prone to high frequency noise, low frequency biases, and scaling factors, among other disturbances (such as non-orthogonality of the axes, eigendynamics of the test masses). Noisy measurements are filtered by the EKF process, but the biases also need to be corrected. For the different sensor models, scaling factors B_i are factored in and biases b_i are added.
The Inertial Measurement Units (IMUs) which house all sensors[6], are not mounted at the center of gravity of the rocket body, and the sensor frame is not aligned to the body frame. Therefore, a transformation to the body-fixed frame must be formulated to model the vectorized sensor measurements. This includes orientation correction with the rotation matrix S_i, and for the accelerometers distance to the c.g. d_i.
All these corrections are determined during hard calibration (S_i, d_i, B_i are found in workshop testing, where some corrections are hard-coded), and by the initialization filter on the launch pad (b_i, see Section 2.5).

[5] Initialization with Euler angles is also possible[1, p. 126], but too much effort to write down here.

[6] Simplified here: an IMU is defined as Gyroscope + Accelerometer, but Magnetometer and Barometer are on the same board/housing.

Angular velocity - Gyroscope

The rate gyroscope measures the angular velocity vector Ω. The measurement is in the rotated sensor frame, and needs to be transformed to the body-fixed frame as[7]

$$\omega = S_\Omega \Omega - b_\Omega \qquad \Omega = S_\Omega^T(\omega + b_\Omega) \tag{I.19}$$

where S_Ω is the rotation vector from body-fixed to the sensor frame. Additionally, the offset biases b_Ω, as determined by the Initilization filter, are corrected for. Note that the the right hand side ($\Omega = ...$) is used in the measurement model.[8] This transformation is done for each IMU separately, with their own $S_{\Omega,i}$ and $b_{\Omega,i}$.

Acceleration - Accelerometer

The accelerometer measures the acceleration vector A. The transformation from the sensor frame to the body frame is computed as [3]

$$a = S_A A - \dot{\omega} \times d_A - \omega \times (\omega \times d_A) \tag{I.20}$$

resulting in the corrected specific force a in the body-fixed frame, with the vector from rocket center of mass to the sensor d_A, and the rotation matrix from the sensor frame to the body frame S_A. This corrects for angular and centrifugal accelerations of a rotating rocket, which show up in points away from the center of gravity but are not “real", i.e. do not change the velocity of the rocket. This transformation is performed for each IMU separately. The resulting velocity derivative in the body-fixed frame is then [1, 3]

$$\dot{v} = a - \omega \times v + Sg^G \tag{I.21}$$

which is the same as Equation I.16, only the measured specific force a has replaced the calculated forces $\mathcal{F}/m$. The problem of whether to use a modelled external force $\mathcal{F}$ or the measured specific force a is addressed in Section 1.5.

Magnetic field - Magnetometer

The magnetometer measures the local magnetic field vector M. This is mostly determined by Earths magnetic field M_E^G (which is assumed to be constant over the flight, and predetermined by initialization on the ground), but can include disturbances from local sources on the rocket and ground support equipment.

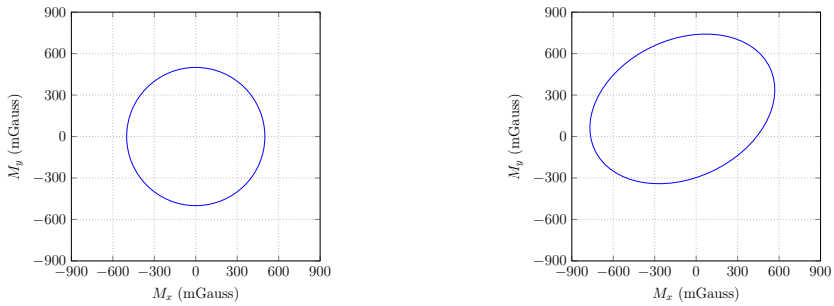
Hard iron disturbances emanate from permanent and electromagnets, which cause an offset bias by vector b_{HI} in the measured field strength as shown in Figure I.3b.
Soft iron disturbances stem from nearby ferromagnetic materials (predominantly Iron, Nickel, Cobalt), which react to the present magnetic field. This causes stretching of the measured field (shown in Figure I.3b) by the matrix B_{SI}, which is unity if no soft iron disturbances are present.

The model of the measured magnetic field with hard and soft iron biases b_{HI} and B_{SI} is

$$M = S_M^T(B_{SI} S M_E^G + b_{HI}) \tag{I.22}$$

[7] In some literature, Ω refers to the skew-symmetric matrix of ω. Here, this matrix is expressed as $\tilde{\omega}$ or $(\omega\times)$, while Ω refers to sensor measurements.

[8] One could use the gyroscope measurement as input for the dynamics model, as is done for the accelerometer, in which case the left hand equation would be used.



(a) Disturbance free (b) Hard and soft iron disturbances

Figure I.3: 2D Visualization of the measured magnetic field [11]

where Earths magnetic field vector M_E^G is rotated into the body-fixed and then the sensor frame with S_M^T. Hard calibration (Section 2.6) must account for both biases and sensor frame rotation. The inverse transformation needed for initialization is

$$M_E^G = S^T(B_{SI}^{-1}(S_M M - b_{HI}) \tag{I.23}$$

Pressure - Barometer

The barometer measures the local air pressure with the sensor offset bias as

$$P = p + b_P \tag{I.24}$$

Air temperature T cannot be measured reliably from barometric sensor, and will therefore not be used. The atmosphere is modelled in Section 1.3.

Canard angle - shaft encoder

Somewhere in the gearing between servo actuator and canard surface, there is a rotational encoder on the shaft. The encoder measures the canard angle directly, and the resulting encoder signal δ_e is

$$\delta_e = \delta \tag{I.25}$$

Any biases and scaling is calibrated for by the encoder handler, and the resulting signal is just noisy.

Satellite navigation

Position in 3D as well as ground speed and heading could be determined by GPS. However, the nominal trajectory goes well outside operational limits of most (or all) commercial GPS receivers (being $|a| < 4.5$g, alt < 18km, $|v| < 515$m/s), the altitude and velocity limits are imposed by the US International Traffic in Arms Regulations. While a GPS may be useful for recovery purposes, it cannot be used during most of the ascent, and therefore the filter will not include GPS.

1.3 Atmosphere and air data

The static pressure p, temperature T, density ρ, and the local speed of sound c of the surrounding air vary with altitude, as shown in Figure I.4. To predict the barometer

measurements and the aerodynamic effects, it is important to determine these variables accurately. The air at altitude l is modelled according to the US standard atmosphere [12] as

$$T = T_B - k_B(l - l_B) \tag{I.26}$$

$$p = p_B \exp\left(\frac{-g(l - l_B)}{RT_B}\right) \qquad p = p_B \left(\frac{T_B}{k_B(l - l_B)}\right)^{\frac{-g}{Rk_B}} \tag{I.27}$$

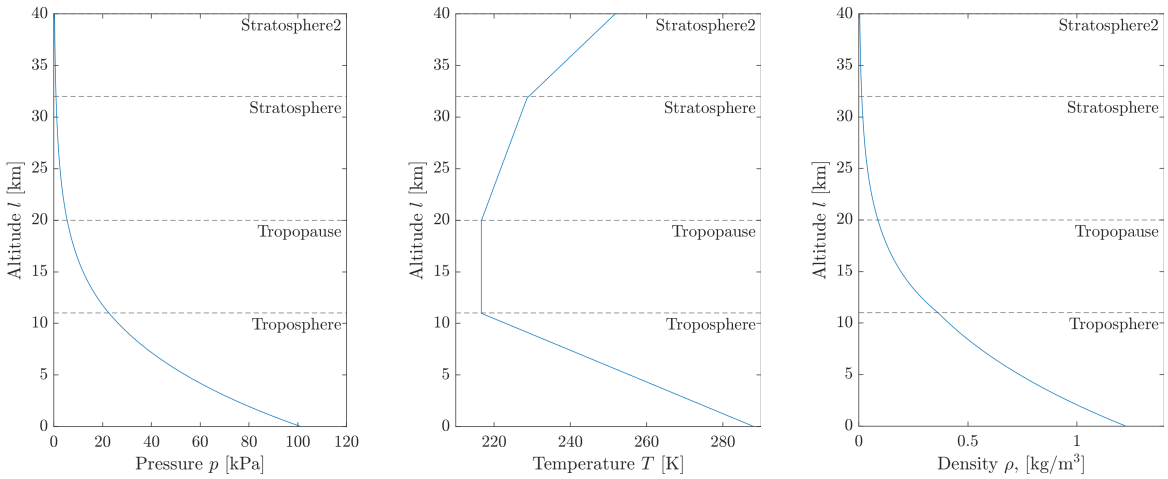
$$\rho = \frac{p}{RT} \tag{I.28}$$

$$c = \sqrt{\gamma RT} \tag{I.29}$$

where the index B indicates a base value from Table I.1. The adiabatic index of air is $\gamma = 1.4$, and the specific gas constant of air is $R = 287.0579\frac{\mathrm{J}}{\mathrm{kgK}}$. To calculate the static air pressure, Equation I.27 is split in two, where the left equation is only used for a zero lapse rate $k_B = 0$.

Layer	Altitude l_B [m]	Pressure p_B [Pa]	Temperature T_B [K]	Lapse rate k_B $[\frac{\mathrm{K}}{\mathrm{m}}]$
Troposphere	0	101325	288.15	0.0065
Tropopause	11000	22632.1	216.65	0
Stratosphere	20000	5474.9	216.65	-0.001
Stratopause	32000	868.02	228.65	-0.0028

Table I.1: Atmosphere base data



(a) Pressure (b) Temperature (c) Density

Figure I.4: Air value variations with altitude, according to the US standard atmosphere model [12].

Dynamic pressure & Mach number

The airspeed of the rocket is defined as $V_A = |v|$, which assumes no or only little wind disturbances[9]. The dynamic pressure is defined as

$$\bar{p} = \frac{1}{2}\rho V_A^2 \tag{I.30}$$

[9]These are difficult to predict, and will thus act as disturbances on the rocket-estimator loop.

which is the predominant factor in the aerodynamic forces and moments. As the local speed of sounds varies with altitude, the Mach number is $\mathrm{Ma} = V_A/c$. The Mach number has influence on the aerodynamic coefficients, and the location of the center of pressure.

Angle of attack & sideslip

The attitude of the rocket relative to the oncoming airstream is defined with the *angle of attack* α and the *angle of sideslip* β. As any wind gusts act as disturbances, both α and β are defined from the velocity vector of the rocket in the body-fixed frame, as seen in Figure I.5. The angles are therefore

$$|v_x| \geq 0 \qquad\qquad |v_x| \leq 0$$

$$\alpha = \arctan(v_z/v_x) \qquad\qquad \alpha = \pi - \arctan(v_z/v_x) \tag{I.31}$$

$$\beta = \arctan(v_y/v_x) \qquad\qquad \beta = \pi - \arctan(v_y/v_x) \tag{I.32}$$

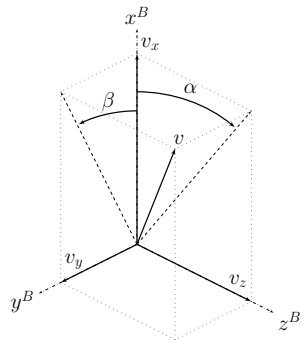


Figure I.5: Diagram of the aerodynamic angles α and β in the body-fixed coordinate frame

1.4 Forces and moments

The rocket dynamics have external forces and torques acting on the body, whcih are described here. Note that the forces are described here, but the filter uses accelerometer data in the dynamics model making the force computations unnecessary. The model could also be used not as a filter, but e.g. in a flight simulation, where the forces would be used. The external torques are still important however, as they couple some states to the angular velocity measurements.

Aerodynamic forces and moments

Most of the external forces and torques acting on the rocket will stem from aerodynamics. The effect of rocket body geometry, including fins and nosecone, can be approximated using Barrowman's method [13].

Any aerodynamic force for each component is defined as

$$\mathcal{F}^i = \bar{p}\, C^i_{\dots} A \tag{I.33}$$

where the aerodynamic coefficient (lift C_L, normal C_N, drag C_D, roll C_l) and reference area A depend on the geometry of the component, and the lift coefficient encompasses information about the angle of attack.

The torque produced by the aerodynamic forces is determined by the vector of the aerodynamic forces, acting through the the *center of pressure* and producing a moment around the center of gravity as

$$\mathcal{T}^i = \mathcal{F}^i \times d^i \tag{I.34}$$

where d^i is the vector from the center of pressure to the center of gravity (direction matters!). Here ^{i} indicates an arbitrary component (it is not an exponent).

The total aerodynamic forces and moments are

$$\mathcal{F}_A = \sum_i \mathcal{F}^i \qquad\qquad \mathcal{T}_A = \sum_i \mathcal{T}^i \tag{I.35}$$

with the coefficients listed in Table I.2.

Symbol	Value	Name
$C_{L_\delta}^{\text{can}}$ or C_L		Lift coefficient, canards
L_δ		Roll control derivative
$C_{l_{\omega_x}}^{\text{fins}}$		Roll damping coefficient, fins
L_{ω_x}		Roll damping derivative, fins
C_{N_α}		Normal force coefficient, entire rocket

Table I.2: Aerodynamic parameters

Aerodynamics of the canards and roll moments

The coefficient of lift is approximated using thin airfoil theory [12]. The canards have a low aspect ratio of about $0.5 \lesssim AR \lesssim 1$, and thin airfoil theory thus predicts a lift coefficient for each canard of [12]

$$C_L^{\text{can}} = \frac{\pi}{2} AR\, \delta \approx 1.5\, \delta \tag{I.36}$$

for small angles of attack $\delta < 40°$ of the canard surfaces. The resulting force of one canard is $\mathcal{F}_1^{\text{can}} = \bar{p}\, C_{L_\delta}^{\text{can}} \delta A_1^{\text{can}}$, which is orthogonal to the x^B vector, and acts on the center of pressure of the canard surface.

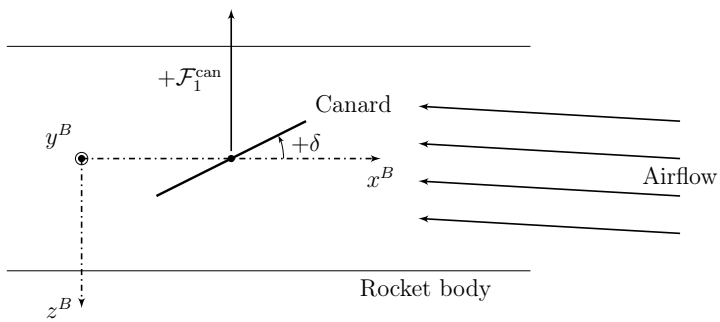


Figure I.6: Diagram of the canard angle δ in the body-fixed coordinate frame. Modelled after [8].

As the canards are of equal size, and move in equal and opposite angles, the normal force on the rocket body cancels out (for small angles of attack of the rocket body), it remains

a net moment about the rocket x-axis. The forces and moments produced by the canards are thus

$$\mathcal{F}^{\text{can}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \mathcal{T}^{\text{can}} = \begin{bmatrix} \bar{p}\, C_{L_\delta}^{\text{can}} \delta A^{\text{can}} d_y^{\text{can}} \\ 0 \\ 0 \end{bmatrix} \tag{I.37}$$

where A^{can} is the total area of all canards and d_y^{can} is the distance between the rocket roll axis x^B and the center of pressure of one of the canards. The instantaneous roll acceleration of the canards can be written with the roll control derivative L_δ as

$$\dot{\omega}_x = \mathcal{T}_x^{\text{can}} / J_x = L_\delta\, \delta \qquad L_\delta = \bar{p}\, C_{L_\delta}^{\text{can}} A^{\text{can}} d_y^{\text{can}} / J_x \tag{I.38}$$

which simplifies notation of the roll dynamics as shown in Section 3.1.

The roll forces of the rocket body are dominated by fin effects, namely roll forcing from misalignment, and roll damping from the fin wake of a rolling rocket. The roll forcing is difficult to predict if the fin misalignment is not exactly know, so any forcing will be treated as disturbances acting on the rocket. The roll forcing is therefore neglected, but the roll damping is

$$\mathcal{F}^{\text{fins}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \qquad \mathcal{T}^{\text{fins}} = \begin{bmatrix} \bar{p}\, C_{l_{\omega_x}}^{\text{fin}} \omega_x A^{\text{fins}} d_y^{\text{fins}} \\ 0 \\ 0 \end{bmatrix} \tag{I.39}$$

The instantaneous roll acceleration from roll damping of the fins can be written with the roll damping derivative L_{ω_x} as

$$\dot{\omega}_x = \mathcal{T}_x^{\text{fins}} / J_x = L_{\omega_x}\, \omega_x \qquad L_{\omega_x} = \bar{p}\, C_{L_{\omega_x}}^{\text{fins}} A^{\text{fins}} d_y^{\text{fins}} / J_x \tag{I.40}$$

Aerodynamic normal forces and moments

Barrowman's method can be employed to derive the location of the center of pressure and the normal force coefficient, for each component of the rocket body [13, 14]. Accurate parameters are determined by the Flight Dynamics subsystem.

Important effects are aerodynamic drag, the total normal force and moment, the roll control and roll damping coefficients, and their respective centers of pressure.

The normal forces and the resulting pitch/yaw torques are

$$\mathcal{F}^{\text{normal}} = \begin{bmatrix} 0 \\ \bar{p}\, C_{N_\alpha} \alpha A^{\text{ref}} \\ \bar{p}\, C_{N_\beta} \beta A^{\text{ref}} \end{bmatrix} \qquad \mathcal{T}^{\text{normal}} = \begin{bmatrix} 0 \\ \bar{p}\, C_{N_\alpha} \alpha A^{\text{ref}} d^{\text{ref}} \\ \bar{p}\, C_{N_\beta} \beta A^{\text{ref}} d^{\text{ref}} \end{bmatrix} \tag{I.41}$$

Propulsion

The forces and moments produced by the engine are determined by the Propulsion subsystem. As we cannot predict thrust misalignment, it is assumed zero with any misalignment acting as disturbances, so the moment produces by the engine is zero. Any sloshing or pogo-effect is too complex to model here, and will be neglected. This leaves an axial propulsion force as some function of the engine performance F_{engine}, so

$$\mathcal{F}_P = \begin{bmatrix} \mathcal{F}_{\text{engine}}(t, \rho, \ldots) \\ 0 \\ 0 \end{bmatrix} \tag{I.42}$$

Rail constraints

The rocket is attached to the launch rail for the first few meters of altitude, constraining the rocket to one degree of freedom (translating upwards). This effect only acts during the first few fractions of a second, and will therefore be neglected for the flight. However, as long as the rocket is (knowingly) on the launch rail, the movement constraints can be used to initialize the state, and to determine sensor biases.

1.5 Estimator model equations

The rocket model is made up of algebraic and differential equations [1, 2, 3], represented in *state-space* as [2, 3] (Equations I.1 and I.2)

$$\dot{x} = f(t, x, u)$$
$$y = h(t, x, u)$$

with the state x, measurement output y, and control input u. Here the state equation f and the output equation h are nonlinear functions.

States x

From textbooks [1, 3, 4], lectures [5], papers [6, 7], the industry standard for choosing states of aeronautical vehicles is using attitude angles, attitude rates, velocity, and position.

Here, position is reduced to altitude only, as longitude and latitude are not required and cannot be measured without GPS[10]. The entire state vector is then

$$x = \begin{bmatrix} q & \omega & v & l & C_L & \delta \end{bmatrix}^T \tag{I.43}$$

with the attitude represented as quaternions $q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^T$, the angular velocity in the body-fixed frame $\omega = \begin{bmatrix} \omega_x & \omega_y & \omega_z \end{bmatrix}^T$ (body rates)[11], the velocity in the body-fixed frame $v = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix}^T$, and the altitude over sea level l. Additionally, the canards have the coefficient of lift C_L as we are unsure about the aerodynamic behaviour of the canards (and as a state, the coefficient can be estimated), and the angle-of-attack δ which is used to carry first order actuator dynamics.

Measurements y

The measurements arrive in the measurement vector

$$y = \begin{bmatrix} \Omega & M & P & \delta_e \end{bmatrix}^T \tag{I.44}$$

with measured body rates from the rate gyroscope $\Omega = \begin{bmatrix} \Omega_x & \Omega_y & \Omega_z \end{bmatrix}^T$, the magnetic field measurement $M = \begin{bmatrix} M_x & M_y & M_z \end{bmatrix}^T$ (which can be transformed to attitude [12]) from the magnetometer, additionally the static air pressure P is provided by the barometer (the temperature T is not used, as the measurement is unreliable). The encoder signal δ_e also

[10] Commercial GPS lockouts are expected at our speed and altitude, to prevent guided missile design.

[11] In literature the body rates are often depicted as $\omega_x = p$, $\omega_y = q$, and $\omega_z = r$. As this may be confusing, we'll stick with ω_i for the rates in this documentation.

[12] The inverse transformation is not unique as it is invariant about the field direction, but we can't use the accelerometer in free flight to find the gravity vector

arrives in the measurement vector. The accelerometer data A will be provided to the filter in the input vector, as this simplifies calculations.[13] If multiple sensors are used for each measurement, they will be stacked in the vector as $y = \begin{bmatrix} \Omega_1 & \Omega_2 & M_1 & M_2 & P_1 & P_2 & T_1 & T_2 \end{bmatrix}$.

Inputs u

The control input vector is the commanded canard angle to the actuator δ_u and, as stated in the previous paragraph, the accelerometer data $A = \begin{bmatrix} A_x & A_y & A_z \end{bmatrix}^T$, which may also be stacked.

As the acceleration is not a state, and cannot be directly computed from the other states (aerodynamic modelling is too indirect), it is used in the state prediction step instead of the correction step. The measured acceleration A enters the prediction function as a control input. As these measurements do not enter the filter in the classical way, their weighting of the expected covariances has to happen in the prediction model. The transformation to the body-fixed body-centered frame from Equation I.20 is performed first, after which the a_i will be weighted and then fed to Equation I.21.

$$u = \begin{bmatrix} \delta_u & A \end{bmatrix}^T \tag{I.45}$$

The actual canard angle is represented as the state δ, to account for actuator dynamics. The computation from desired canard angle to servo command should be performed in the controller. If multiple IMUs are used, the measurements are stacked in the input vector as $u = \begin{bmatrix} \delta_u & A_1 & A_2 & A_3 \end{bmatrix}$.

Dynamics model

$$\dot{\hat{x}} = f(t, \hat{x}, u)$$

predicts the time derivative of the state vector, $\dot{\hat{x}}$ (Equation I.43), using the control input u (Equation I.45), and the following model equations:

- Attitude: I.11
- Rates: I.18
- Velocities: I.21 for filtering and I.16 for simulation
- Position/Altitude: I.17
- Canard coefficient: 1st order dynamics $\dot{C}_L = -(1/\tau_c)(C_L - C_{L,\,\text{theory}})$
- Canard angle: 1st order dynamics $\dot{\delta} = -(1/\tau_\delta)(\delta - \delta_u)$

Measurement model

$$\hat{y} = h(t, \hat{x}, u)$$

predicts the sensor outputs $\hat{y}$ from the current state estimate $\hat{x}$, and the model equations:

- Gyroscope: I.19
- Accelerometer: used in state prediction as I.20 and I.21
- Magnetometer: I.23

[13] This is quite common practice

- Barometer: I.26 to I.29

1.6 Controller model equations

The model used for controller synthesis must be linear, time-invariant, and controllable. Linear time-invariant state-space models can be derived by linearizing the equations f and h around some design point (t_0, x_0, u_0), which results in the equations (Equations I.3 and I.4)

$$\dot{x} = A(x - x_0) + B(u - u_0)$$
$$y = C(x - x_0) + D(u - u_0)$$

These equations may be employed around a collection of design points $\{t_s, x_s, u_s\}$ do account for varying parameters of the matrices (A, B, C, D).

As the model must be controllable, it makes some simplifications compared to Section 2. This is because the actuating canards can only produce a roll torque, i.e. a moment about the x-axis, but cannot directly influence other states, e.g. pitch and yaw rates. This results in the full model from Section 1.5 not being controllable, i.e. we cannot use the pole placement method to arbitrarily choose our desired dynamics. Additionally, the

The cross coupling terms are neglected, and the the attitude is reduced from four quaternions to just the roll angle. This roll angle ϕ is computed from entries of the rotation matrix as $\phi = \text{atan2}(S_{23}, S_{33})$. The velocity and altitude show up in the dynamic pressure $\bar{p}$. The dynamic pressure and the canard coefficient C_L are the scheduling variables of the controller. The concocted state x_R for the roll model is

$$x_R = \begin{bmatrix} \phi & \omega_x & \delta \end{bmatrix}^T \tag{I.46}$$

The resulting state-space model is then

$$\dot{x}_R = Ax_R + B\delta_u \qquad\qquad y_R = Cx_R \tag{I.47}$$

with the matrices

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & L_{\omega_x} & L_\delta \\ 0 & 0 & -\frac{1}{\tau} \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ 0 \\ 1/\tau \end{bmatrix} \qquad C = \begin{bmatrix} I_3 \end{bmatrix} \tag{I.48}$$

where the roll control derivative is $L_\delta(\bar{p},\, C_L)$, a function of dynamic pressure and the canard coefficient of lift. Roll damping is modelled by the roll damping derivative $L_{\omega_x}(\bar{p},\, C_{L\omega})$. The 1st order actuator dynamics are accounted for with the δ state, and are characterized by the time constant τ (the value should be determined by experimental system identification). Any gear ratio between servo motor and canards may be expressed as the numerator of the bottom entry of B (here set to 1). The matrix C combines the states of the system to an output signal, and can be set arbitrarily for response analysis. For the case $C = \begin{bmatrix} I_3 \end{bmatrix}$, the output signal is the entire state vector $y(t) = x(t)$, for the case $C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$, the ouput signal is $y(t) = x_1(t) = \phi$.

2 Estimator

The state estimation algorithm is responsible for accurately determining the current rocket state. The rocket state is a set of data points which describe the condition of the rocket dynamics, and is made up of the rocket attitude, velocity, altitude, aerodynamic coefficients (coefficients are not quite certain due to the lack of available windtunnel testing) and other air data. As the underlying dynamics are nonlinear in some cases, an *Extended Kalman Filter* (EKF) is used to estimate the current state. The EKF algorithm uses these modelled dynamics to

1. predict the current state from the last state and control inputs (desired canard angles) to the rocket, and then
2. correct the state using measurements from onboard sensors, including rate gyroscopes, magnetometers, accelerometers, barometers.

This whole process is a *Bayes filter*, where the state estimate has an associated belief in the accuracy of the estimate. Each iteration of the filter uses the the priori (old) state estimate and belief, to compute/update the posteriori (new) estimate and belief.

2.1 Filter algorithm

The EKF thus propagates a state estimate $\hat{x}$ in discrete time steps, along with the belief in form of the covariance matrix P. The matrix represents the "uncertainty" of the state estimate and their correlation, i.e. higher values represent more uncertainty.

For its algorithm the EKF uses the continuous-time prediction model f and the discrete-time measurement model h, as well as their Jacobians

$$F(x,u) = \frac{\partial f}{\partial x} \qquad H(x) = \frac{\partial h}{\partial x} \tag{I.49}$$

which can be understood as the *sensitivity* of the nonlinear equations f and h to variations of the state. Essentially, the Jacobians indicate how much the outputs of the functions f and h change for small changes of each state variable. As the the filter is computed at sufficiently short (see Section 2.4) time intervals, the state does not change greatly between each computation step. The functions f and h can be approximated by linearization, and the covariance is propagated using the resulting Jacobians.

One complete iteration of the algorithm of the Continuous-Discrete EKF is [2, 15, 6] [14]

Predict:

solve IVPs $\left(\hat{x}_k^- \ , \ P_k^-\right)$ with $\hat{x}(t-T) = \hat{x}_{k-1}^+ \ , \ P(t-T) = P_{k-1}^+$

$$\dot{\hat{x}}(t) = f(\hat{x}, u_{k-1}) \tag{I.50}$$

$$\dot{P}(t) = F(\hat{x}, u_{k-1})P(t) + P(t)F^T(\hat{x}, u_{k-1}) + Q_e \tag{I.51}$$

Correct:

$$K_k = P_k^- H^T(\hat{x}_k^-)\left[H(\hat{x}_k^-)P_k^- H^T(\hat{x}_k^-) + R_e\right]^{-1} \tag{I.52}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k\left[y_k - h(\hat{x}_k^-)\right] \tag{I.53}$$

$$P_k^+ = \left[I - K_k H(\hat{x}_k^-)\right] P_k^- \left[I - K_k H(\hat{x}_k^-)\right]^T + K_k R_e K_k^T \tag{I.54}$$

[14] This is not the cleanest formulation, but hopefully understandable. See [2, pp. 259] for more thorough explanations of the algorithm.

where the measurements are used for the state correction, while the control input is used for prediction.

Here the prediction part consists of the continuous-time dynamics (Equation I.50) and the associated covariance in continuous-time (Equation I.51). These dynamics and the solution of the previous filter iteration ($\hat{x}(0) = \hat{x}_{k-1}^{+}$ and $P(0) = P_{k-1}^{+}$) yield an *Initial Value Problem* (IVP). The discrete solutions $\hat{x}_k^-$ and P_k^- of the IVP have to be computed by integration. As the IVP is solved in each filter iteration, the integration bounds are the timestamps of the previous and current iteration.

The correction part computes the filter gain (or Kalman gain) K_k using the covariance of the estimate P and the sensitivity of the measurement model H (Equation I.52). As the measurement model is discrete, the computation of the predicted measurements ($\hat{y} = h(t, \hat{x}_k^-)$) and the correction of the state estimate (Equation I.53) is straightforward. The covariance of the estimate is updated (Equation I.54) using the Joseph stabilized form, to ensure the covariance remains positive semidefinite, even if the gain K_k is "suboptimal" [2].

The EKF is tuned using the weighting matrices Q_e and R_e, which represented the expected noise covariances of plant and measurement (see Section 2.3).

Observability

To check if the filter can converge at a given state, the linearized system is examined for observability. The observability matrix (with the number of states $n = 13$) is

$$\mathcal{O} = \begin{bmatrix} H \\ HF \\ HF^2 \\ \vdots \\ HF^n \end{bmatrix} \tag{I.55}$$

which has rank 12 for the given model, i.e. the system is not fully observable. The reason is the lack of an absolute measurement in attitude, as the magnetometer measurement is invariant for rotations about the magnetic field vector. This will result in slow attitude drift from the true value, which is still an improvement over simply integrating the rate gyroscopes as the magnetometer constrains the attitude in the other two degrees of freedom.[15]

Initialization

The EKF is initialized by setting the state estimate and the covariance estimate to

$$\hat{x}(0) = \bar{x}_0 \qquad\qquad P(0) = \bar{P}_0 \tag{I.56}$$

where $\bar{x}_0$ can be computed while the rocket is stationary on the rail, and $\bar{P}_0$ is the uncertainty of the initial state (often quite low). Determining accurate initial values is treated in Section 2.5.

[15] For vehicles in straight and level movement, like ships, cars, and even airplanes, this is corrected by accelerometers measuring the gravity vector. However, the rocket will be in free fall the entire flight and measuring gravity is not possible using accelerometers.

2.2 Optimal estimation theory

This section is here for completeness, it is not relevant for implementation. The system has continuous-time dynamics and discrete-time measurements [2]

$$\dot{x} = f(x, u, t) + w(t) \tag{I.57}$$

$$y_k = h(x_k, k) + v_k \tag{I.58}$$

where $w(t)$ is the process noise and v_k is the measurement noise, which are additive, uncorrelated, and white zero-mean Gaussian processes [2, 15]. Both processes satisfy [15] [16]

$$E\left[w(t)w^T(t+\tau)\right] = Q_e\delta(t) \qquad E\left[v_k v_{k+1}^T\right] = R_e\delta(k) \tag{I.59}$$

The estimation error $\varepsilon = x - \hat{x}$ is to be minimized in the cost function with weight vector ϱ [15]

$$\min V = \lim_{t\to\infty} E\left[\varepsilon^T \varrho \varrho^T \varepsilon\right] \tag{I.60}$$

for which the solution is of the form [15]

$$V^* = \varrho^T P \varrho \tag{I.61}$$

In infinite time, P satisfies the filter algebraic Ricatti equation [15]

$$PF + F^T P - PH^T R_e^{-1} HP + Q_e = 0 \tag{I.62}$$

This results in the estimator feedback [2]

$$\tilde{x} = K_k\left[y - h(\hat{x}_k)\right] \tag{I.63}$$

where the gain K_k is computed with the a-priori covariance estimate P_k^-

$$K_k = P_k^- H_k^T \left[H_k P_k^- H_k^T + R_e\right]^{-1} \tag{I.64}$$

(since the posteriori covariance P_k is dependent on the current gain, the implicit equation is $K_k = P_k^+ H_k^T R_e^{-1}$).

Due to the nonlinear behaviour of the state space functions, both P and K_k must be computed in real time as new measurements become available [2]. It is to note that the Kalman filter generally is only optimal for the mathematical model, not the actual plant (in this case the rocket) itself [2]. Additionally, the EKF is just quasi-optimal, as optimality is generally only proven for the case of linear state prediction and measurement functions.

2.3 Filter tuning

As seen in Equation I.59, the tuning matrices Q_e and R_e represent the expected noise magnitudes of the state process and the measurements. Higher values in the matrices indicate a higher noise magnitude, i.e. lower trust in the accuracy of the measurement. The EKF is be tuned by adjusting the entries of these matrices, which must be positive semi-definite. The matrices can be filled to account for cross-covariance between the states or measurements (e.g. higher altitude or higher speed lowers accuracy of pressure measurement). However, this is often difficult to characterize, and the tuning matrices are often filled diagonally as initial guesses [16].

[16] Prof. Werner please forgive the butchering of the theory that is to follow

Typically, the values in R_e represent the noise behaviour of the sensors, where the the entries can be taken directly from sensor datasheets.

The matrix Q_e can be adjusted according to the expected “noise" in the state prediction process. Here, the entries can represent expected disturbances of the rocket flight, and again higher values indicate higher expected disturbances and thus lower trust in the accuracy of the state prediction.

Closed-loop stability depends on the robustness margins of the combined plant-estimator-controller loop. As shown in [17], using a state observer inside the control loop *could* lead to vanishing margins. This can lead to instability, if the plant models do not match the reality perfectly. To recover some of the robustness margins of the optimal controller (see Section 3.3), a loop transfer recovery method is employed [16]

$$Q_e \quad \rightarrow \quad Q_e + \sigma B_e \tag{I.65}$$

where the weighting matrix Q_e has some added, fictitious noise σ acting on the control input channel. Here $B_e = \frac{\partial f}{\partial u}$ is the sensitivity of the input signal - but only for the actual control input, not for the accelerometer measurement (so B_e is all zero, except a single one in the δ row, δ column). In practice, this means that for higher weights in the δ/δ field of Q_e, the closed-loop robustness margins improve [16].

2.4 Implementation notes

The resulting state is passed to the control algorithm.

Numerics

The Jacobians F and H are computed numerically. Numerical methods for finite differentiation are shown in Section 4.1.

For the prediction step, numerical solvers are implemented to compute the solutions of the IVPs. In this case these methods are *Runge-Kutta(4)* for the state prediction and *Heun's method* for the state covariance (see Section 4.2). The methods are picked as a trade-off between accuracy and computational effort. Computation time is an important factor in implementing the state estimation, as the discrete EKF process should be performed at $\omega_s \gtrapprox 50 \cdot \omega_c$ [16], with $\omega_s = 200Hz$ to avoid the rocket crossover region, which is in frequencies around $\omega_c \approx 4Hz$.

Quaternions

To ensure a stable quaternion behaviour, the quaternion has to be normed to 1 after each operation, including the correction step. This means setting

$$q = q/\text{norm}(q) \qquad \text{or} \qquad x(1:4) = x(1:4)/\text{norm}(x(1:4)) \tag{I.66}$$

in the prediction model and after the correction step $\hat{x}_k^+ = \hat{x}_k^- + K_k \left[y_k - h(\hat{x}_k^-)\right]$, respectively.

2.5 Initializor

This algorithm serves to compute accurate values for $\bar{x}_0$, and the sensor biases b. The Initializor is a filter which runs in the Estimator loop while the rocket is stationary on the launch rail, instead of the EKF. Shortly before liftoff, the Initializor hands over the latest state estimate ($\bar{x}_0$) and the determined sensor biases to the EKF. To get accurate values for the sensor biases, the Initializor runs a lowpass filter with all the current sensor data z_k and the already filtered data s_k (start with $s_0 = \mathbf{0}$)

$$s_{k+1} = s_k + \alpha(z_k - s_k) \tag{I.67}$$

This should take place over a long time frame (longer than a minute) to attenuate any noise effects, or movement through plumbing or wind. The gain α is chosen according to the allowable/expected time frame of initialization, a lower α results in a smoother, but slower converging signal (α is the time constant of this exponential discrete-time filter, the resulting shape is $e^{\alpha\, dt}$ with sampling time dt).

The IMUs should also be at a constant temperature, which must be the same as in flight (though some IMUs perform internal correction of the gyroscope temperature). The biases and earth magnetic field are combined to the bias vector b.

As the filter runs while the rocket is stationary on the launch rail, several assumption can be made:

1. The velocity of the rocket v will be zero, and thus the altitude l is constant.
2. The acceleration of the rocket $\dot{v}$ will be zero.
3. The body rates ω will be zero, and thus the attitude q is constant.

With the filtered data s, the state can be computed using the sensor models as follows:

1. The accelerometer measurement will be the rotated gravity vector, so $S_A A = S^T g^G$. No bias can be set for the individual axes, but the magnitude of the accelerometer measurement (part of s_k) should be g.
2. The gyroscope bias b_Ω can be set to the filtered angular velocity (part of s_k) while the rocket is stationary.
3. Altitude can be computed using the filtered pressure (part of s_k), according to the (inverted) atmosphere model. As the launch altitude is is known, any bias offset of the barometer b_P can be corrected for.
4. The initial attitude will be close to, but not exactly upright. It is defined as yaw/pitch only, and thus can be computed uniquely from the accelerometer measurement.
5. The magnetic field vector of the Earth M_E^G can be computed from the filtered magnetic measurement M (part of s_k) and by inverting the estimated initial attitude.

2.6 Hard calibration

The orientation of all sensor frames (S_Ω, S_A, S_M) to the rocket body must be determined *well before* launch day. Additionally, the position of the accelerometers d_A to the dry center of gravity, and the the hard (b_{HI}) and soft (B_{SI}) iron biases for the magnetometer correction should be determined.

3 Controller

The controller is responsible for driving stabilizing the rocket state, attenuating any disturbances, and driving it to some desired value. In their configuration the canards can only provide a torque around the roll (x) axis of the rocket. This imposes some constraints on the controller design, as the whole state is under-actuated.

The controller chosen for the canards is linear state feedback, where the feedback gains are computed by the *Linear Quadratic Regulator* (LQR) method [16, 15]. The current state is determined by the state estimation algorithm (the Estimator), as seen in Figure I.7. The loop is a two-degree-of-freedom design, where the reference signal is prescaled by a reference gain. To account for the non-linearity of the rocket dynamics the gains are selected from a gain schedule, which is a look-up table of gains for different parameters of the linear model from Equations I.68, I.69, and I.70.

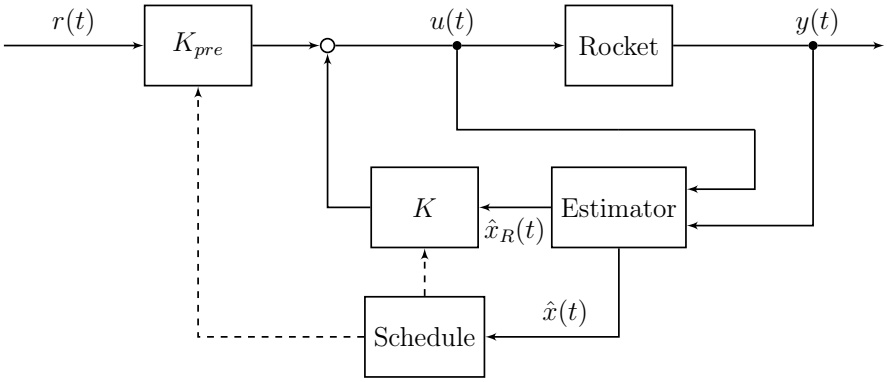


Figure I.7: Block diagram of the control loop. 2-DOF estimator-based state feedback [16] with gain scheduling.

3.1 Controller roll model

As the model must be linear, time-invariant, and controllable for the LQR synthesis, it makes some simplifications compared to Section 2. The cross coupling terms are neglected, and the the attitude is reduced from four quaternions to just the roll angle. This roll angle ϕ is computed from entries of the rotation matrix as $\phi = \text{atan2}(S_{23}, S_{33})$. The velocity and altitude show up in the dynamic pressure $\bar{p}$. The dynamic pressure and the canard coefficient C_L are the scheduling variables of the controller. The concocted state x_R for the roll model is

$$x_R = \begin{bmatrix} \phi & \omega_x & \delta \end{bmatrix}^T \tag{I.68}$$

The resulting state-space model is then

$$\dot{x}_R = Ax_R + B\delta_u \qquad\qquad y_R = Cx_R \tag{I.69}$$

with the matrices

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & L_{\omega_x} & L_\delta \\ 0 & 0 & -\frac{1}{\tau} \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ 0 \\ 1/\tau \end{bmatrix} \qquad C = \begin{bmatrix} I_3 \end{bmatrix} \tag{I.70}$$

where the roll control derivative is $L_\delta(\bar{p},\, C_L)$, a function of dynamic pressure and the canard coefficient of lift. Roll damping is modelled by the roll damping derivative $L_{\omega_x}(\bar{p},\, C_{L\omega})$. The 1st order actuator dynamics are accounted for with the δ state, and are characterized by the time constant τ (the value should be determined by experimental system identification). Any gear ratio between servo motor and canards may be expressed as the numerator of the bottom entry of B (here set to 1). The matrix C combines the states of the system to an output signal, and can be set arbitrarily for response analysis. For the case $C = \begin{bmatrix} I_3 \end{bmatrix}$, the output signal is the entire state vector $y(t) = x(t)$, for the case $C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$, the ouput signal is $y(t) = x_1(t) = \phi$.

The (approximate in continuous time) closed loop is

$$\dot{x}_R = (A + BK)x_R + BK_{pre}r \qquad\qquad y_R = Cx_R \tag{I.71}$$

To determine if pole placement is possible, which is a necessary condition for LQR [16], the roll model is checked for controllability. The controllability matrix is

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} \tag{I.72}$$

which has full rank for all $L_\delta \neq 0$, i.e. there exists a feasible feedback gain for every flight condition, except for zero velocity ($\|v\| = 0$) or a zero coefficient of lift ($C_L = 0$).

3.2 Control law and tuning

To compute the desired canard angle $u = \delta_u$, the the current rocket state as determined by the state estimation is used. The control law is a state feedback, combined with a feedforward part for the reference as

$$u(t) = K\hat{x}_R(t) + K_{pre}r(t) \tag{I.73}$$

where the gains $K = \begin{bmatrix} K_\phi & K_{\omega_X} & K_\delta \end{bmatrix}$ are determined by performing the LQR method around a linear model

$$K(\hat{x}) = -\texttt{lqr}(A(\hat{x}), B, Q_c, R_c) \tag{I.74}$$

with Matlab's `lqr` function. For the discrete controller, the function $\texttt{lqrd}(A, B, Q, R, dt)$ is be used to directly account for discrete dynamics with sampling time dt, but using a continuous-time system (A,B). To note: for fast sampling times (much higher sampling frequency than the bandwidth of the model), the feedback gains of `lqr` and `lqrd` become approximately equal. The method is tuned by adjusting the weighting matrices Q_c and R_c. As shown in section 3.3, the matrix Q_c contains the weights for penalizing the state error, while the matrix R_c weighs the expended control effort.

The scalar reference gain K_{pre} is computed for a steady state gain of 1, as

$$K_{pre}(\hat{x}) = \frac{-1}{C\left[A(\hat{x}) + BK(\hat{x})\right]^{-1} B} \tag{I.75}$$

where C depends on the variable to be tracked. With $C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$, the channel $r \to \phi$ is selected, which results the roll angle tracking the reference. The channel $r \to \omega_x$ could also be selected via $C = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$, this would result in tracking of the roll rate.

The gains K and K_{pre} can be precomputed for each combination of parameters (some slowly varying states) of $A(\hat{x})$. As gain scheduling is used to account for slowly-varying parameters, the gains are be computed for each design point x_s, and combined to a look-up table (see Section 3.4). This way the implemented controller does not need to compute the gains with the LQR method, but can just select the gains from the gain scheduling table.

3.3 Optimal control theory

This section is here for completeness, it is not relevant for implementation. The LQR method uses the weighting matrices Q and R to solve the linear quadratic optimization problem [15] [17]

$$\min V = \int_0^\infty (x^T(t)Qx(t) + u^T(t)Ru(t))dt \tag{I.76}$$

for which the solution is of the form

$$V^* = x^T P x \tag{I.77}$$

In infinite time, P satisfies the algebraic Ricatti equation

$$PA + A^T P - PBR^{-1}B^T P + Q = 0 \tag{I.78}$$

which results in the control law

$$u^*(t) = Kx(t) \tag{I.79}$$

$$K = -R^{-1}B^T P \tag{I.80}$$

The resulting controller for the linear system is optimal in the trade-off of minimizing the state error $x(t)$ and control effort $u(t)$ [16]. The closed-loop is guaranteed to be stable [15], and has margins of $[\infty, 0.5]$ in gain and $60°$ in phase. This guarantee is only valid however for linear systems with full state measurements [17]. The control law is still useful for controlling nonlinear and observed systems, as the robustness properties can be partially preserved [15].

3.4 Gain scheduling

As the models from canard angle to roll rate are nonlinear and highly dependent on some slowly-varying states, a gain-scheduling scheme will be implemented. The design points x_s of the schedule are characterized by these slowly-varying states, being as air density, velocity, and aerodynamic coefficients of the canards. The air density and velocity are combined to the dynamic pressure $\bar{p}$. The simplified model is evaluated around these design points using the current state estimate (so $A(\hat{x})$), to form the matrix A. As the LQR method produces guaranteed robustness margins [17, 16, 15], the interpolation between two close design points should always produce a stable control loop.

A large variety of design points are chosen, both along the nominal flight profile, but also encompassing a larger flight envelope to accommodate off-nominal trajectories. Design points for the condition L_δ are omitted from the tuning process. The control gains for all design points are computed beforehand and stored in a look-up table. In flight, the control algorithms interpolate the gains from the look-up table according to the current

[17] I think this came out better than the estimation theory

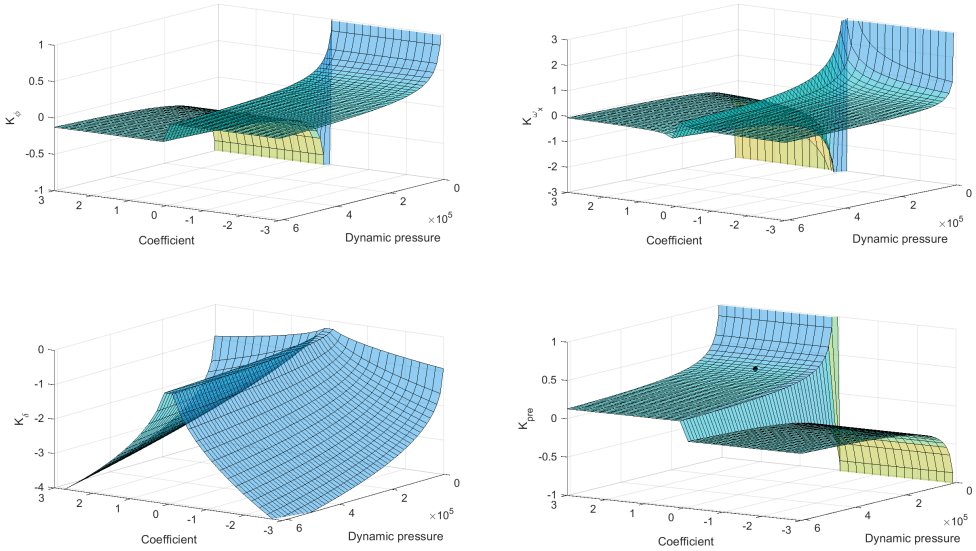


Figure I.8: Scheduling surfaces along dynamic pressure $\bar{p}$ and canard coefficient of lift C_L. Feedback gains K_ϕ, K_p, K_δ and reference gain K_{pre}.

state estimate. The pre-computed feedback gains for the roll model are shown in Figure I.8, plotted over the scheduling variables $\bar{p}$ and C_L. To note is the discontinuity at $\bar{p} = 0$ and especially at $C_L = 0$, where no feasible design points exist. The feedback gain are interpolated over this split.

3.5 Response analysis

To analyze some aspects of the control loop, the closed-loop responses are analyzed. The closed-loop is technically a sampled-data system, with continuous time (the rocket) and discrete time (estimator and controller) dynamics. For a high sampling rate (short time periods between control outputs), the continuous-time closed-loop is approximately (from Equation I.71).

$$\dot{x}_R = (A + BK)x_R + BK_{pre}r \qquad\qquad y_R = Cx_R$$

with $C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$ for the $r \to \phi$ channel.

The closed-loop step response for a collection of parameters (here varying velocities) is shown in Figure I.9.

The Bode plots of the closed-loop for a collection of parameters (here varying velocities) is shown in Figure I.10.

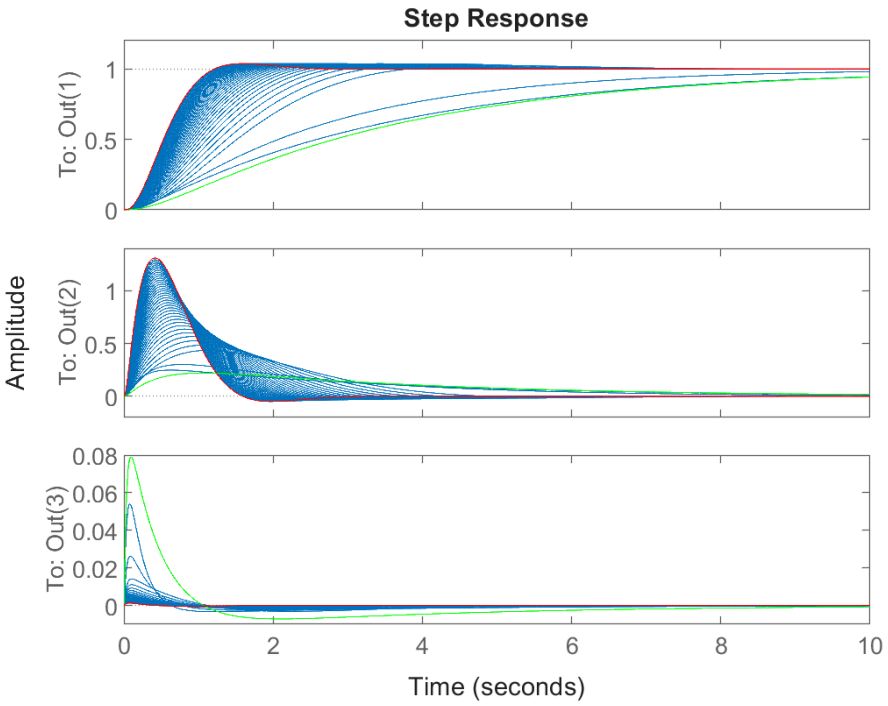


Figure I.9: Step responses of the closed-loop system for velocities between 30m/s (green) and 800m/s (red), $l = 1000$m, $C_L = 1.5$. Outputs: 1:ϕ, 2:ω_x, 3:δ.

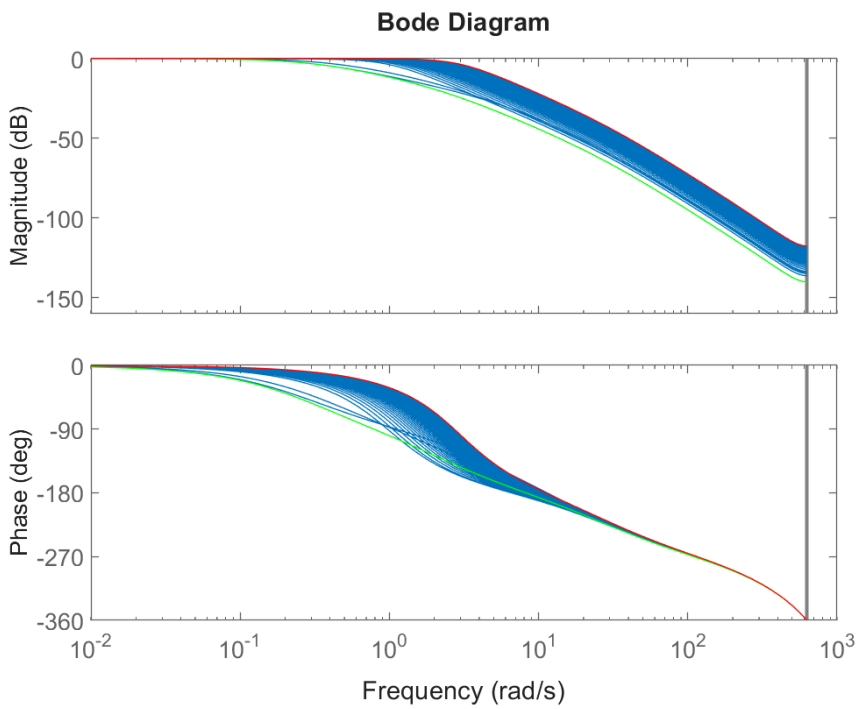


Figure I.10: Frequency response of the closed-loop system for velocities between 30m/s (green) and 800m/s (red), $l = 1000$m, $C_L = 1.5$. The response is cut off above the sampling frequency of the processor.

4 Numerical methods

4.1 Numerical Jacobian computation

The Jacobian J_x is the matrix of all first-order partial derivatives of a vector-valued function. For the prediction and measurement model, the Jacobians of the functions f and h ([18]) are needed with respect to the state x (where $n = \text{length}(x)$). The Jacobian of any function with respect to x is defined as [18]

$$J_x = \frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \tag{I.81}$$

Jacobians may be derived analytically from the function, but this is often cumbersome and prone to errors. Therefore, it is common practice to compute the matrix numerically. A frequently used method is using *finite differences*, however this is known to be not very efficient nor accurate, and has issues with numerical instability [18].

Complex-step differentiation

A more efficient way to calculate Jacobian matrices numerically is to use *complex step differentiation*. In this method with step size ε, the k-th column of the matrix (J_{x_k}) is [19]

$$J_{x_k} = \text{Im}\left(\frac{f(t, x + \mathrm{i}\varepsilon e_k, u)}{\varepsilon}\right) \tag{I.82}$$

where e_k is the k-th unit vector in the state-space (i.e. $e \in X$, and $e_1 = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}^T$, $e_2 = \begin{bmatrix} 0 & 1 & \cdots & 0 \end{bmatrix}^T$, etc.). This must be computed for each entry of x, so that the Jacobian is

$$J_x = \begin{bmatrix} J_{x_1} & J_{x_2} & \cdots & J_{x_n} \end{bmatrix} \tag{I.83}$$

The evaluation of the functions with a complex state vector $\begin{bmatrix} x_1 + \mathrm{i} & x_2 & \cdots & x_n \end{bmatrix}^T$ seems very unintuitive, but works for every holomorphic $\mathbb{R}^n \to \mathbb{R}^m$ function[19] [18]. Complex-step differentiation also is numerically stable, and has also an $\mathrm{O}(h^2)$ error size [19]. Both finite differences and complex-step scale linearly with the number of variables, but the finite difference method requires two function evaluations per variable, while complex-step requires only one.

4.2 Numerical Integration methods

To get a new discrete state from the continuous-time prediction process of the EKF, the nonlinear differential equations I.50 and I.51 have to be solved. As the last state estimate is available as an initial value, this can be expressed as the *Initial Value Problem* (IVP)

$$\dot{x}(t) = f(t, x, u) \qquad\qquad x(0) = x_n \tag{I.84}$$

which is can be solved by numerical integration. The integration methods need to be explicit, as the filter in the prediction step has only past state information.

[18] The following equations are shown for an "f", but are of course the same for f and h

[19] for implementation this requires that the function can be evaluated with complex inputs

Explicit Runge-Kutta method

The explicit s-stage *Runge-Kutta* method is compactly written as [20]

$$x_{n+1} = x_n + \varepsilon \sum_{i=1}^{s} c_i k_i \tag{I.85}$$

$$k_i = f\left(t_n + a_i \varepsilon\,,\; x_n + \varepsilon \sum_{j=1}^{i-1} b_{ij} k_j\,,\; u\right) \tag{I.86}$$

with the time step size ε, and where a_i, b_{ij}, and c_i are the entries of the vectors a and c and the matrix B, displayed in a *Butcher tableau*

$$\begin{array}{c|c} a & B \\ \hline & c^T \end{array} \tag{I.87}$$

These can be very large, as in the *Dormand-Prince* method (RK45, which is the standard solver in Matlab). For our cases we will keep the tableaus small.

The second order ($s = 2$) butcher tableau of the Heun method [20], which is used for the prediction covariance, is

$$\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array} \tag{I.88}$$

The fourth order ($s = 4$) butcher tableau of the classical Runge-Kutta method RK4 [20], which is used for solving the state prediction, is

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{I.89}$$

5 Implementation

5.1 Overview

- Estimator: computes a state x = `estimator_state_t` from IMU measurements, control inputs, and encoder measurements
- Controller: computes a desired servo angle $u = \delta_u$ = `controller_output_t` from the estimated state

5.2 Estimator

Propagates an estimation of the rocket state: x = `estimator_state_t`

Structs and co.

- The state: x = `estimator_state_t`
- The covariance: P = `estimator_covariance_t`

Initializor

Runs repeatedly after initialization message has been sent, and until `flight_phase` detects injector valve opening.

Tasks in each iteration:

- Determine gyroscope biases
- Calculate launch attitude with accelerometer measurements
- Calculate the Earth magnetic field vector from magnetometer measurements
- Determine launch altitude with atmosphere model
- Set initial state to hand over to EKF

The initializor uses an iterative lowpass filter to filter out the sensor noise:
`mean(k+1) = mean(k) + time_const*(measurement-mean(k));`

EKF

Runs repeatedly after injector valve opening.

Tasks in each iteration:

- Integrate state from last state using dynamics model equations
- Integrate covariance matrix from last state using filter ricatti equation
- Calculate filter gain from covariance matrix
- Predict IMU measurements using measuremen model equations
- Calculate new state from feedback: filter gain * (actual - predicted IMU measurements)
- Calculate new covariance matrix

5.3 Model functions

5.4 Controller

6 Symbols

Short	Expansion	Description
EKF	Extended Kalman Filter	Extension of Kalman filter to nonlinear systems
IMU	Inertial Measurement Unit	Packaged gyroscope and accelerometer sensor
IVP	Initial Value Problem	Set of ODEs with given inital values, to be solved
LQR	Linear Quadratic Regulator	Optimal controller for linear systems
LTI	Linear time-invariant	State-space model with linear dynamics, do not depend explicitl

Table I.3: Acronyms

Name	Dimension	Description
x	$n\times1$	State
$\hat{x}$	$n\times1$	State estimate
u	$m\times1$	Control input (controller $\rightarrow$ rocket)
y	$l\times1$	Measurement output (rocket $\rightarrow$ estimator)
$f(x,u)$	$n\times1$ function	Dynamics function
$h(x)$	$l\times1$ function	Measurement function
$F(x,u)$	$n\times n$ function	Dynamics Jacobian
$H(x)$	$l\times n$ function	Measurement Jacobian
$B(x,u)$	$n\times m$ function	Input Jacobian
P	$n\times n$	Covariance estimate
K_k	$l\times n$	Filter gain
Q_e	$n\times n$	State process covariance
R_e	$l\times l$	Measurement covariance
K_s	$m\times n$	Feedback gain
Q_c	$n\times n$	State error weighting
R_c	$m\times m$	Control effort weighting

Table I.4: Estimator and Controller variables

Name	Dimension	Unit	Description
q	4		Quaternion
ω	3	$\frac{\text{rad}}{\text{s}}$	Angular velocity in body-fixed frame (body rates)
v	3	$\frac{\text{m}}{\text{s}}$	Velocity in the body-fixed frame
r^G	3	m	Position in the inertial (geographic) frame
l	1	m	Altitude
S	3×3		Rotation matrix, body-fixed $\rightarrow$ inertial
S_i	3×3		Rotation matrix, sensor $i \rightarrow$ body-fixed
d_i	3	m	Distance vector, sensor $i \rightarrow$ center of gravity
b_i	3		Bias of sensor i
Ω	3	$\frac{\text{rad}}{\text{s}}$	Measured body rates
M	3	mT	Measured magnetic field
A	3	$\frac{\text{m}}{\text{s}^2}$	Measured acceleration
P	1	Pa	Measured air pressure
T	1	K	Measured air temperature
m	1	kg	Mass
J	3×3	kgm^2	Moment of inertia
$\mathcal{F}$	3	N	External force in the body-fixed frame
$\mathcal{T}$	3	Nm	External torque in the body-fixed frame
g	3	$\frac{\text{m}}{\text{s}^2}$	Gravitational acceleration
T	1	K	Air temperature
ρ	1	$\frac{\text{kg}}{\text{m}^3}$	Air density
p	1	Pa	Static pressure
$\bar{p}$	1	Pa	Dynamic pressure
C_i	1		Aerodynamic coefficient
A_i	1	m^2	Aerodynamic reference area
d_i	1	m	Distance vector, center of pressure $i \rightarrow$ center of gravity

Table I.5: Important modelling variables

Chapter II

Rocket flight Simulation

Joe Dolina, Tristan Yan-Klassen

7 Plant Model

7.1 Background

2023-2024, Borealis, and Airbrakes

Borealis was the team's first attempt at flying (and for all intents and purposes, developing) active control systems, in the form of airbrakes. At the outset of the airbrakes project, it was envisioned that a working group would develop something like Fig II.1. Incidentally, it was "envisioned" that we would be flying TVC stabilization in 2026.

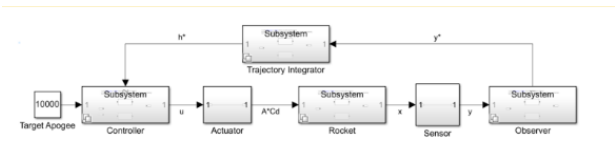


Figure II.1: Airbrakes Simulink structure

Everything except "Actuator" and "Rocket" were implemented to some degree in the final flight software. However, the most useful output from Simulink that year was Fig II.1 itself.

A PID controller was ultimately selected for airbrakes because nobody had the knowledge or experience to do anything fancier. This controller was "tuned" using a plugin for OpenRocket [21], essentially going in and dynamically modifying the drag force used by Openrocket in its calculations with values produced from external computational fluid dynamics (CFD) simulations. Later versions included the ability to inject random noise into the state measurements. Under these noisy conditions, simulations suggested the controller could get within about 20m of the target altitude over most of the controllable regime (20,000 +/- 2,000 ft.).

While much more useful at the time than having no controller-in-loop simulation options at all, the OpenRocket plugin was not sustainable. The overlap between Waterloo Rocketry members (and budding control engineers) and Java developers was and still is very small.

The final version ran the controller as a binary compiled from the same C source as the board on the rocket, and while this was also a promising development in the team's testing practices, the process for doing this was even less accessible than running the plugin itself. The plugin project was ultimately an uphill battle to turn OpenRocket into something it wasn't meant to be.

This brings us to the present design cycle. The controls subsystem has flight heritage in the form of the processor and motor control boards, it has experienced members (in this author's opinion, some of the very best members), and it even has its own WEEF account. Given the sophistication of controller and state estimator design we are targeting this year for canards, as well as the broad interest from controls members in Matlab and Simulink, there is a strong justification to pursue the development of a complete rocket plant model in Simulink this year.

Project Jupiter

Projeto Jupiter is a rocketry team from the University of São Paulo, Brazil. They conveniently already developed most of what is necessary for a plant model of a sounding rocket with canards [22]. Rather than trying to start more or less from scratch as it was planned last year, the goal this year is essentially to modify their plant model with our rocket design parameters, verify it for uncontrolled flight, and then begin simulating the effects of canards.

7.2 Theory

Compared to a dynamic model of a real launch vehicle, a sounding rocket is considerably simpler. In particular, sloshing (source: Thomas), bending, and TVC dynamics do not need to be considered. The rocket can be treated as a single rigid body with time-varying mass and mass distribution. The only forces acting on an uncontrolled rocket are engine thrust, gravity, and drag. Of these, gravity applies no moment by definition, and the moment applied by thrust, due to misalignment with the rocket center of mass, is considered negligible for our purposes. Since gravity is well defined and engine thrust is defined externally from static fire test data, this leaves aerodynamic drag as the most complicated aspect of simulation.

Barrowman provides empirical methods for computing the center of pressure of a rocket's components at relatively low Mach numbers. (link FD docs). The coefficient of drag (C_d) is modeled only as a function of Mach number for a given rocket, using OpenRocket's methodology, of which this author is unfamiliar. In the future, other drag models could certainly be considered.

7.3 Implementation

Everything is in metric - MKS. Due to restrictions of certain blocks, the WGS84 earth model is used where applicable.

Matlab Scripts

A rocket-specific script (e.g. Borealis.m) hard-codes the geometric parameters like reference length, fin location, etc. This script also imports a .csv file from OpenRocket containing

relevant data. The Borealis script includes some logic to turn the C_d and Mach number time series into a lookup table. Barrowman.m takes the rocket parameters and evaluates the Barrowman equations based on the reference geometry.

Top Level Plant

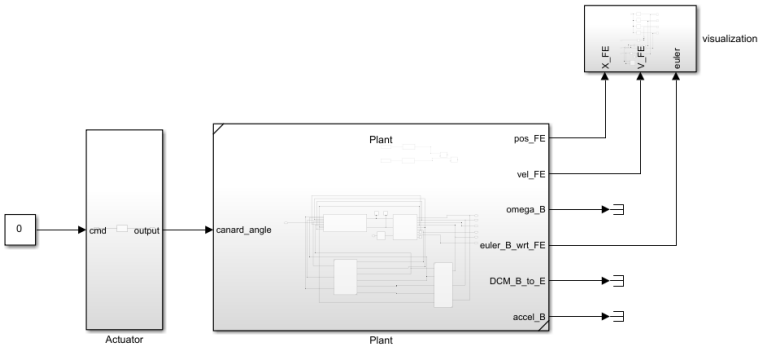


Figure II.2: Canards Plant Model Interface

The only explicit input to the plant is the deflection angle of the canards. The plant model outputs the full 6DOF state in a combination of reference frames, and the direction cosine matrix (DCM) from flat earth to body frame. It also outputs the translational acceleration, in order to model accelerometer dynamics.

The actuator subsystem implements a first order system representing the response delay of the servo.

6DOF Variant Mass Model

The core of the plant model is the "Simple Variable Mass 6DOF" block provided by the Aerospace Blockset. It takes in net force and moment about the center of mass at each time step and outputs translational and rotational position and velocity. Note that the inputs are provided in the body fixed frame, with the X axis coincident with the longitudinal axis of the rocket body. The translational position and velocity are provided in the flat earth reference frame. The block doesn't output mass vs time, instead giving you starting mass, ending mass, and a $\frac{dm}{dt}$ input. You gotta dig in there and log the signal or hope and pray. But it seems to work as expected so far.

Wind Model

The wind simulation operates similar to Niskanen. The winds take on a constant speed at all altitudes, with pink noise for turbulence added on top. This turbulence is also responsible for any and all rotational speed of the wind. Additionally, gusting has been added on top of the base wind model, which allows for gusts of set strength and direction to happen after a specified time. Similar to OpenRocket, the simulation takes a user inputted base wind velocity, base wind direction, and turbulence value. The user must also specify the magnitude, time of occurrence, and duration of gusts.

Roll Dynamics

For roll dynamics, the canards and fins are split into separate parts and treated independently. This allows the canards to be handled with more detail or with different aerodynamic models if needed.

For the fins, the aerodynamic coefficients and derivatives are calculated using Barrowman's equations and then fed into the Simulink model. This is done by the "roll forcing and damping block" in the plant. The only significant change from an aerodynamics standpoint is the mach-speed correction term on the coefficients, which is normally taken to be

$$\mathcal{B}^{-1} = \frac{1}{\sqrt{|1 - M^2|}}$$

Since this breaks at $M = 0$ and it is commonly held that it peaks at around 2 near sonic speeds, we take $\mathcal{B}$ to be the minimum of the above expression and 2.

Normal Force Components

The methodology used to compute the normal force components is the same as that of OpenRocket [23] (see Sec. 3.2.1). The rocket body is divided into its components: nosecone, body, fin, tail, and canards (if applicable). The local velocity of the rocket at the component center of pressure, relative to the wind, is computed and along with the component C_{N_α} is used to compute the normal force and moment vectors. This functionality is encapsulated in a generic "lift component" block, whose internals are shown in Fig II.3:

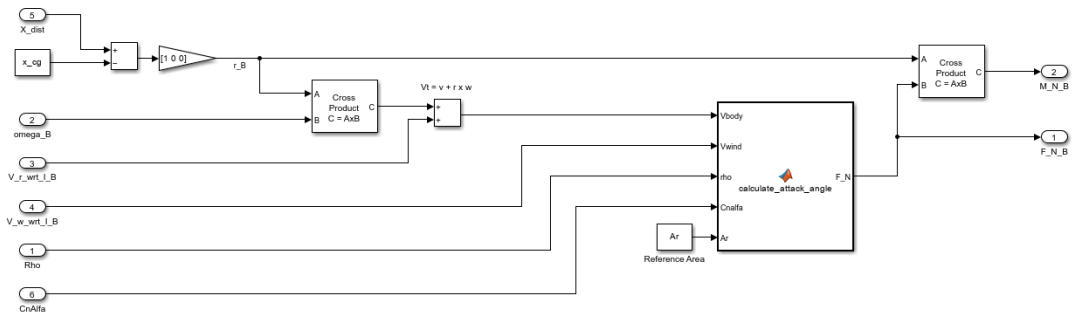


Figure II.3: Lift Component subsystem

These blocks are composed to calculate the net normal force and moment:

Simulated Sensor Outputs

In order to test the robustness of the controller to real-world sensor outputs, the IMU (accelerometer, gyroscope), magnetometer, and barometer (only pressure) outputs are simulated, adding noise and bias. This is done through custom MatLab scripts that track drift over the simulation and continuously add it to the measured quantities.

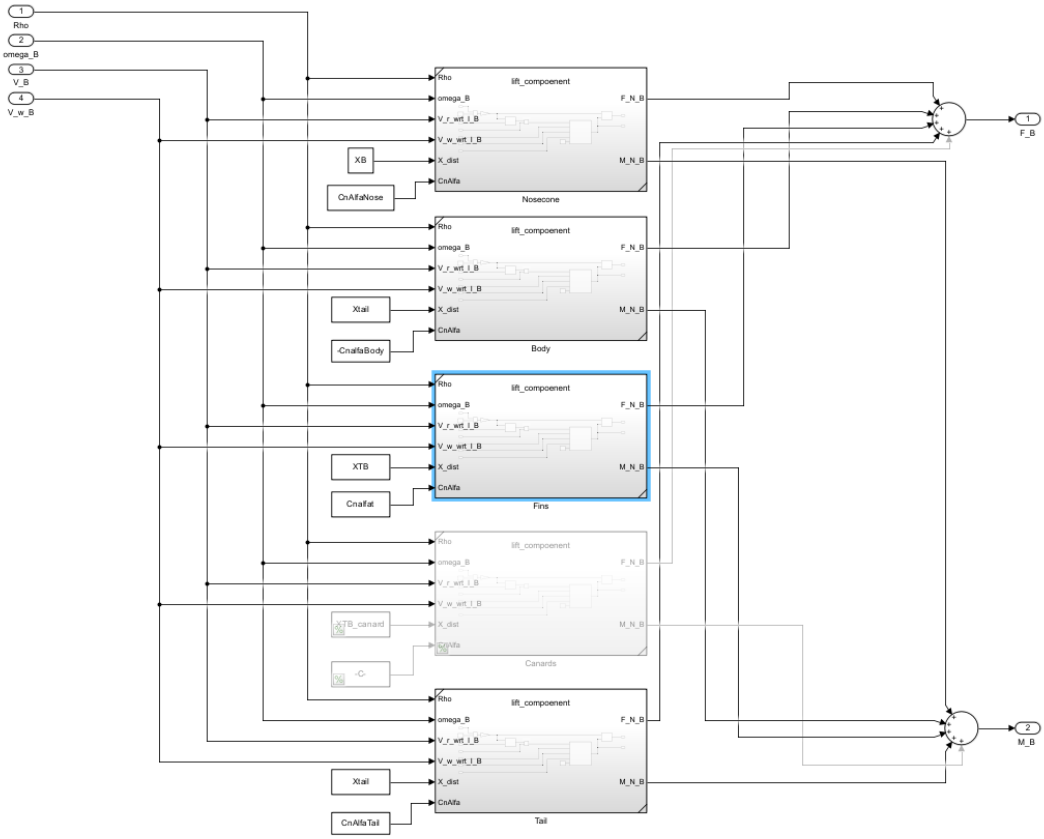


Figure II.4: Normal Force and Moment

8 Simulation results

8.1 Rocket flight

Comparison with OpenRocket

8.2 Estimator in the loop

8.3 Controller in the loop

8.4 Full closed-loop

References

[1] Peter H. Zipfel. *Modeling and Simulation of Aerospace Vehicle Dynamics*. 2nd. Reston: American Institute of Aeronautics and Astronautics, 2007. ISBN: 978-1-56347-875-8. DOI: 10.2514/4.862182.

[2] Frank L. Lewis, Lihua Xie, and Dan Popa. *Optimal and Robust Estimation*. 2nd. CRC Press, Taylor & Francis Group, 2008. ISBN: 978-0-8493-9008-1.

[3] Brian L. Stevens, Eric N. Johnson, and Frank L. Lewis. *Aircraft Control and Simulation: Dynamics, Controls Design, and Autonomous Systems*. Third edition. Hoboken, New Jersey: Wiley, 2015. ISBN: 9781119174882. DOI: 10.1002/9781119174882.

[4] Ashish Tewari. *Advanced control of aircraft, spacecraft and rockets*. Aerospace series. Hoboken, N.J: Wiley, 2011. ISBN: 1119971209. DOI: 10.1002/9781119971191.

[5] Julian Theis. *Flight Control Law Design: Lecture Notes*. Hamburg University of Technology, Institute of Aircraft Systems (FST), 2023.

[6] Bryan Tong Minh. "Real-time position and attitude determination of the Stratos II sounding rocket". MA thesis. Delft University of Technology, 2012.

[7] Julian Theis et al. "Linear Parameter-Varying Feedforward Control: A Missile Autopilot Design". In: *AIAA Guidance, Navigation, and Control Conference 2015*. Red Hook, NY: Curran, 2015. ISBN: 978-1-62410-339-1. DOI: 10.2514/6.2015-2001.

[8] George M. Siouris. *Missile Guidance and Control Systems*. New York, NY: Springer, 2004. ISBN: 978-0-387-00726-7. DOI: 10.1007/b97614.

[9] Joan Solà. *Quaternion kinematics for the error-state Kalman filter*. Technical University of Catalonia, Institut de Robòtica i Informàtica Industrial, 2017.

[10] Werner Schiehlen and Peter Eberhard. *Technische Dynamik*. 5th. Springer Vieweg, 2017. ISBN: 978-3-658-18456-8. DOI: 10.1007/978-3-658-18457-5.

[11] *VectorNav Inertial Navigation Primer*. 2024. URL: https://www.vectornav.com/resources/inertial-navigation-primer/ (visited on 12/07/2024).

[12] Robert F. Stengel. *Flight Dynamics*. Princeton, NJ: Princeton University Press, 2004. ISBN: 9781400866816. DOI: 10.1515/9781400866816.

[13] James S. Barrowman. "The practical calculation of the aerodynamic characteristics of slender finned vehicles". M.S. USA – Washington, D.C.: The Catholic University of America, 1967. URL: https://ntrs.nasa.gov/api/citations/20010047838/downloads/20010047838.pdf.

[14] Sampo Niskanen. "Development of an Open Source model rocket simulation software". M.S. Finland – Helsinki: Helsinki University of Technology, 2009. URL: https://openrocket.sourceforge.net/thesis.pdf.

[15] Herbert Werner. *Optimal and Robust Control: Lecture Notes*. Hamburg University of Technology, Institute of Control Systems, 2021.

[16] Herbert Werner. *Control Systems Theory and Design: Lecture Notes*. Hamburg University of Technology, Institute of Control Systems, 2021.

[17] J. Doyle. "Guaranteed margins for LQG regulators". In: *IEEE Transactions on Automatic Control* 23.4 (1978), pp. 756–757. ISSN: 0018-9286. DOI: 10.1109/TAC.1978.1101812.

[18] Joaquim R. R. A. Martins and Andrew Ning. "Engineering Design Optimization". In: (2021).

[19] Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. "The complex-step derivative approximation". In: *ACM Trans. Math. Softw.* 29.3 (Sept. 2003), pp. 245–262. ISSN: 0098-3500. DOI: 10.1145/838250.838251. URL: https://doi.org/10.1145/838250.838251.

[20] David Griffiths and Desmond Higham. *Numerical Methods for Ordinary Differential Equations: Initial Value Problems*. Jan. 2010. ISBN: 978-0-85729-147-9. DOI: 10.1007/978-0-85729-148-6.

[21] *OR Airbrake Plugin*. https://github.com/waterloo-rocketry/or-airbrake-plugin.

[22] *Project Jupiter Control Canards*. https://github.com/Projeto-Jupiter/CC.

[23] *OpenRocket technical documentation v13.05*. https://openrocket.sourceforge.net/techdoc.pdf.